CS370 Fall 2026: Assignment 1

Instructor: Christopher Batty (email: christopher.batty@uwaterloo.ca)

Due Monday Oct 5th at 11:59pm Eastern time

This assignment consists of written work and coding tasks. Submit all components of your solutions (written/analytical work, code/scripts/notebooks, figures, plots, output, etc.) to Crowdmark in image/PDF form for each question (multiple pages are allowed).

You must ***also*** submit all of your code in *runnable* format (and if necessary, any data needed to reproduce your results) to the last "dummy" question on Crowdmark (Q7), as one or more attachments.

You have a number of options of how to prepare the written part of your answers. You can typeset your solutions, you can write them on a tablet, or you can write on paper and take photos. **It is your responsibility to ensure they are sufficiently legible.** TAs can deduct marks for illegible answers.

Your solutions will be judged not only for correctness but also for the quality of your presentation and explanations. Show your work!

1. **(6 marks)** Floating Point Basics

   (a) **(3 marks)** Convert the decimal (base-10) number $x = 406.889$ into a floating point number system given by $F = \{\beta = 3, t = 7, L = -7, U = 7\}$, using truncation. That is, determine $fl(x)$, expressed in our normalized floating point form.

   (b) **(1 mark)** What is the value of machine epsilon (i.e., unit round-off) E for this system?

   (c) **(1 mark)** What is the positive number with smallest magnitude that can be represented in this system?

   (d) **(1 mark)** If we were to consider changing the allowable exponent range of F, what is the largest value of U for which overflow would occur when performing the conversion described in part (a)?

2. **(10 marks)** Effects of Cancellation

   The roots of a quadratic equation, $ax^2 + bx + c = 0$, can be given by

   $$x_1 = \frac{-b + \sqrt{b \cdot b - 4(a \cdot c)}}{(2a)}, \qquad x_2 = \frac{-b - \sqrt{b \cdot b - 4(a \cdot c)}}{(2a)}$$

   when $a \neq 0$.

   (a) **(3 marks)** Consider the 5-digit arithmetic of a floating point number system $F(10, 5, -10, 10)$ **with truncation**. Using the above standard quadratic formulas and mimicking the behavior of this floating point system, carry out step by step "by hand" (you can use a calculator for the individual computations) the computation of the roots of the following quadratic equation

   $$1.05x^2 + 12.93x + 0.231 = 0. \tag{1}$$

The true roots of the given quadratic, correct up to five significant digits, are -0.017891 and -12.296. What is the *relative* error of each of your computed roots, x_1 and x_2?

(b) **(2 marks)** A *cancellation problem* arises when applying the above formulae for any quadratic equation having the property that

$$|b| \approx \sqrt{b^2 - 4ac}.$$

When this property holds, if $b > 0$ then the quadratic formula for x_1 will exhibit cancellation, while if $b < 0$ then x_2 will exhibit cancellation. Show that a mathematically equivalent formula for x_1 is

$$x_1 = \frac{2c}{-b - \sqrt{b^2 - 4ac}}.$$

Hint: Rationalize the denominator of this expression (i.e., multiply by an appropriate quantity to eliminate the square root in the denominator).

(c) **(2 marks)** The formula for x_2 can be manipulated similarly. Deduce a better algorithm (which avoids cancellation errors) for calculating the roots of a quadratic equation, and present it in the form below.

Algorithm R.

if $b > 0$ then
$x_2 = \frac{-b-\sqrt{b^2-4ac}}{2a}$
$x_1 = \frac{c}{ax_2}$
else
$x_1 =$
$x_2 =$
end

(d) **(3 marks)** Redo the calculation of the roots of equation (1) by applying Algorithm R using the same floating point number system. Compute the relative error of the new computed roots and describe what you observe in comparison to your results from part (a).

3. **(6 marks)** Floating Point Error Analysis

Show that if x is a number (that is exactly represented) in a floating point system F, the relative error of the floating point calculation $(x \otimes x) \ominus 1$ compared to the true value of $x^2 - 1$ is less than or equal to

$$\left(\frac{|x^2|}{|x^2 - 1|} + 1 \right) E \tag{2}$$

assuming we can simply ignore terms proportional to E^2 and higher powers of E, and no overflow/underflow occurs. (Recall that the circled-operator notation indicates a floating point counterpart of a corresponding real arithmetic operator, with our usual guarantee on its relative error. You can assume that the number 1 is also exactly representable in this system.)

4. **(6 marks)** Polynomial Interpolation

Given 2 parameters, u and v, find a cubic polynomial, $p(x)$ such that

$$p(0) = 2,\ p(1) = 6 + 2u + 2v,\ p'(0) = 3v,\ p''(1) = 4u - 2v + 24.$$

Express the coefficients of $p(x)$ in terms of u and v.

5. **(10 marks)** Spline Construction

   (a) **(4 marks)** Write down the linear system of five equations needed to determine the derivative values $s_1,\ s_2,\ s_3,\ s_4,\ s_5$ for the cubic spline $S(x)$ going through the (x, y) points $(-5, 2), (0, 0), (1, -3), (3, 1), (4, 2)$, with a free boundary condition (zero second derivative) at the left end (at $x = -5$), and a prescribed slope of -1 at the right end (at $x = 4$). The final linear system should be given in the form $T\vec{s} = \vec{r}$ where T is a matrix, $\vec{s}$ is the vector of unknown slopes s_i, and $\vec{r}$ is the right-hand-side vector, as described in the course notes, Section 2.5. Show the calculations leading to the entries in your linear system, not just the final system.

   (b) **(6 marks)** In this part, you will prepare a Jupyter notebook (`splineplot.ipynb`) to solve the system you worked out above, determine the spline coefficients, and visualize (plot) the smooth spline over the interval [-5,4], as described more below.

   To solve the system, use numpy arrays to create and represent the matrix and vector you found in part (a) (i.e., "hardcode" the matrix and right-hand-side vector values). Then use the numpy function `numpy.linalg.solve` to compute the solution of the system and thereby find the s_i values.

   Next your code should use the s_i values to determine the coefficients, a_i, b_i, c_i, d_i, for the cubic polynomial on each interval of the spline. The cubic polynomials on the intervals are denoted $S_i(x)$, for $i = 1...4$, and have the following form:

$$\begin{aligned}
S_1(x) &= a_1 + b_1(x+5) + c_1(x+5)^2 + d_1(x+5)^3 && \text{for } x \in [-5, 0] \\
S_2(x) &= a_2 + b_2(x-0) + c_2(x-0)^2 + d_2(x-0)^3 && \text{for } x \in [0, 1] \\
S_3(x) &= a_3 + b_3(x-1) + c_3(x-1)^2 + d_3(x-1)^3 && \text{for } x \in [1, 3] \\
S_4(x) &= a_4 + b_4(x-3) + c_4(x-3)^2 + d_4(x-3)^3 && \text{for } x \in [3, 4].
\end{aligned}$$

   Your code should print out all the coefficients, a_i, b_i, c_i, d_i.

   Finally, produce a single plot of the resulting smoothly connected curve over the domain $[-5, 4]$. This is done by creating a finely spaced set of input x values spanning the domain (e.g., using the numpy function `numpy.linspace`), and for each value, evaluating the appropriate cubic polynomial above to get the corresponding y value. The resulting set of (x, y) data should be plotted using the standard `plot` function (which displays as piecewise linear), but it should visually appear smooth because it was constructed by more finely sampling the smooth spline. Use a different color and marker symbol for each of the four intervals.

   Produce a PDF of your notebook and its results, and submit it as your answer for this part. Also submit the file `splineplot.ipynb` as part of the code submission for this assignment.

6. **(15 marks)** Using Splines for Parametric Curves

   In this question, you will create and display ***parametric curves*** representing a three-letter word or acronym of your choosing in a bubble letter-like style (i.e., smooth block letters), making use of Python's spline and plotting functionality. For example,

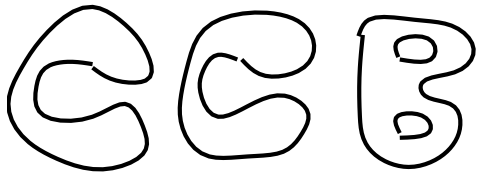


Figure 1: Example of smooth block lettering of your instructor's initials.

Prepare a Jupyter notebook `lettering.ipynb` which does the following:

- Use `matplotlib.pyplot.ginput` to accept mouse-clicks on a blank figure to create a sequence of points (which will become the knots of one continuous parametric spline curve). Try not to use more points than necessary.
- Repeat the above process to collect data for additional distinct curves, and store them into a list of curve data sets. Terminate the input loop when no points are entered (i.e., when the length of the data points returned by `ginput` is 0.)
- For each of the input curve data sets gathered above, do the following...
  - Create a piecewise *linear* plot of the collected set of points. (This should display your word in a jagged straight line form.)
  - Next construct an approximate arc-length parameterization t from the points.
  - Construct two spline objects, for the X- and Y-coordinate vectors separately, based on the arc-length parameterization above, using the SciPy command `make_interp_spline`.
  - Create a finer subdivision of the parameterization. Evaluate the splines at these finer points to approximate the true smooth curve, and plot the resulting parametric curve on a second figure. This curve should represent a single smooth, continuous pen stroke in your bubble font letter.

If your Jupyter environment does not support `ginput`, you may instead construct your point values manually and "hardcode" them in your notebook file.

The output your code produces should be two figures: one *piecewise linear plot* containing all the input point sequences (letters), and a second, separate *smooth spline plot* of all your letters, similar to what is illustrated in Figure 1 above. (Thus for each iteration of the second for loop described above, you should draw on the same plot(s).) As in Figure 1, your curves do need not to close up neatly. Your result should consist of at least 3 distinct (i.e., disconnected) curves, that is, at least one curve per letter.

Produce a PDF of your notebook and its results, and submit it as your answer for this question. Also submit the file `lettering.ipynb` as part of the code submission for this assignment.